

AN INTRODUCTION TO SUPPORT VECTOR MACHINES

ADAM LEWANDOWSKI

ABSTRACT. This is the 'independent study' part of the MA124 Maths by Computer Project at the University of Warwick. This means that this was written well over a year ago, and was meant to be presented in a presentation format. Due to this, it is quite concise, contains a lot of images, and if I were to rewrite it today, it would look significantly different. However, I put a lot of effort into this, and hiding it because I don't like it today would be doing myself from a year ago a disservice.

1. THE IDEA OF SVM'S

In classification problems, we often treat data as a k -dimensional vector, each of a given class, and we search for a $k - 1$ dimensional hyperplane that seperates the points into their respective classes. However, this hyperplane is almost never unique.

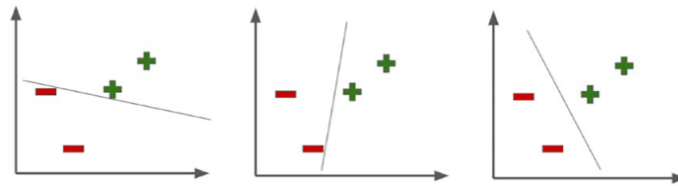


FIGURE 1. The Separating Hyperplane is not unique. [4]

On the left image below, we see clearly why not every seperating hyperplane is built equally. They are far too sensitive. The same goes for the dashed lines on the right - they are too sensitive again. SVM's try find the optimal hyperplane by maximising the distance between *support vectors* - these are the points right on the "edge" of the dataset, closest to the seperating hyperplane. In the image above, they are highlighted in red. This also means that adding datapoints away from the support vectors will not affect the hyperplane.

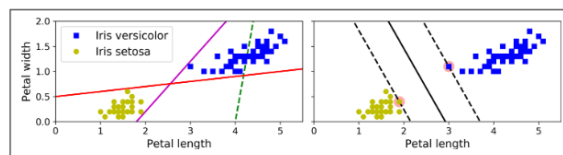


FIGURE 2. The Maximal Separating Hyperplane. [3]

Suppose we are given a linearly separable (meaning a separating hyperplane exists) training dataset of n points of the form $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$, where $\mathbf{x}_i$ is a k -dimensional vector, and $y = \pm 1$ gives the class. We know any hyperplane can be written as the set of $\mathbf{x}$ such that $\mathbf{w}^T \mathbf{x} - b = 0$, where $\mathbf{w}$ is the normal vector to the hyperplane.

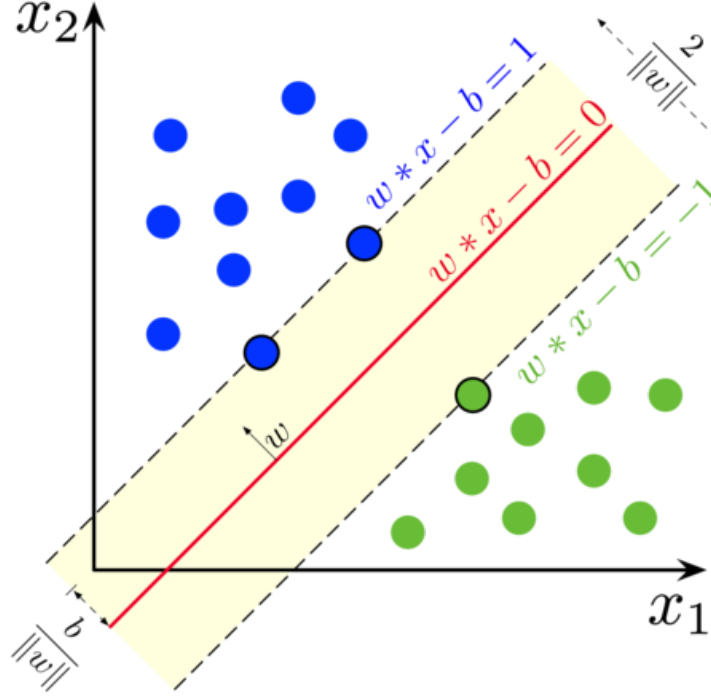


FIGURE 3. The Idea of SVM's. [1]

Looking at the image, we are finding the hyperplane $\mathbf{w}^T \mathbf{x} - b = 0$ that maximises the distance $\frac{2}{\|\mathbf{w}\|}$ subject to each point being on the correct side of the plane. (Notice this is equivalent to minimising $\frac{1}{2} \|\mathbf{w}\|$).

Notice, however, that this under this formulation of the optimisation problem, if there is an outlier in the dataset, then no classifying hyperplane can be found. This can be seen in the left image.

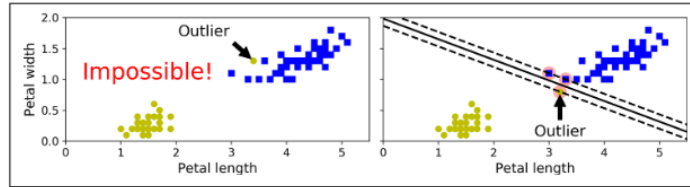


FIGURE 4. Outliers mean no separating hyperplane can exist. [3]

In order to have some classification when the data is not linearly separable we introduce the *hinge loss function*, $L(\mathbf{x}_i, y_i)$, which is zero if our point lies on the correct side of margin, and if not, it's value is proportional to the distance from the margin. We are then trying to minimise $\frac{1}{2} \|\mathbf{w}\| + C$, where $C = \frac{1}{n} \sum_{i=1}^n L(\mathbf{x}_i, y_i)$. We can decide on a maximum value of C which will determine how many margin violations are allowed.

2. THE LINEARLY SEPERABLE CASE

```
from sklearn import svm
from sklearn.model_selection import train_test_split

#Train-test split - 25% of the data
# is reserved for testing natively.

X_train, X_test, y_train, y_test = train_test_split(X,y)

#Training our linear SVM model - one more sensitive to outliers,
#one less so.

svc = svm.SVC(kernel='linear').fit(X_train,y_train)
less_sensitive_svc = svm.SVC(kernel='rbf', C=1, gamma=0.7).fit(X_train, y_train)
#Plotting our test data and the maximal seperating hyperplane.

titles = ['SVC with linear kernel, C=1', 'SVC with RBF kernel']
xx, yy = np.meshgrid(np.arange(X_train[:,0].min()-1, X_train[:,0].max()+1, 0.1),
np.arange(X_train[:,1].min()-1, X_train[:,1].max()+1, 0.1))

Z=svc.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx,yy,Z, cmap=plt.cm.coolwarm, alpha=0.4)
plt.scatter(X_train[:,0], X_train[:,1],c = y_train, marker='o')
plt.title(titles[0])
plt.show()
```

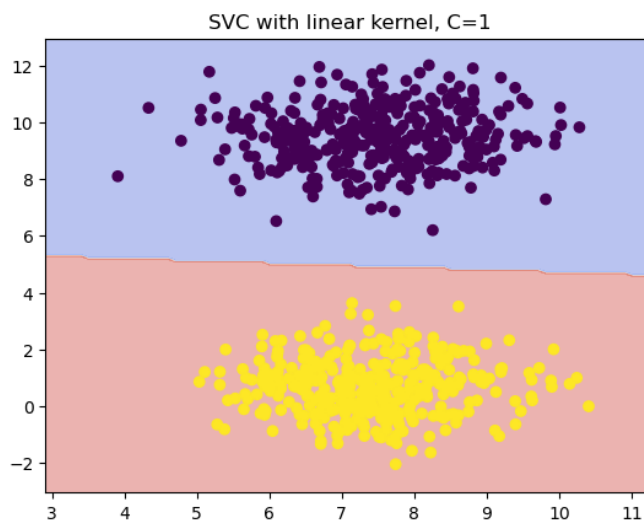


FIGURE 5. The Linearly Seperable Case, illustrated in Python.
(Author's Own)

#The confusion matrix.

```
from sklearn import metrics
import seaborn as sns
y_pred = svc.predict(X_test)
cm = metrics.confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt = ".1f")
```

#As we can see, all points are classified properly.
#Of course, this is an easy dataset.

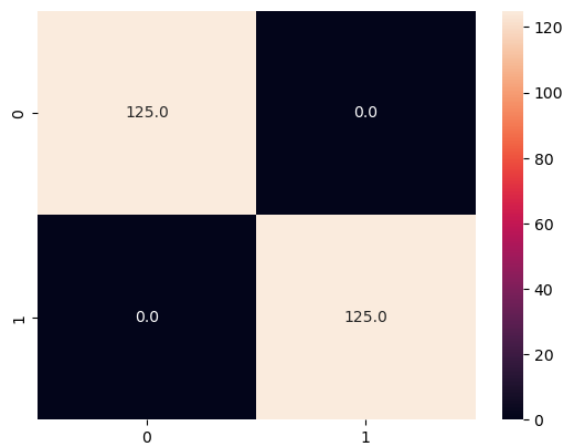


FIGURE 6. The Confusion Matrix for the above dataset. (Author's Own)

3. NON-LINEARLY SEPERABLE DATA

We will look at a few types of classification problems here. First of all, at a binary classification with nonlinear data, and we will see how SVM's deal with that. Then we will look at multilabel classification, in easier and harder cases, and see how SVM's deal.

First of all, binary classification with non-linearly seperable data.

```
from sklearn.datasets import make_moons
#Creating a binary classification problem with non-linearly seperable data

X1, y1 = make_moons(1000, noise=0.2)

plt.scatter(X1[:,0], X1[:,1], c=y1)
plt.show()
```

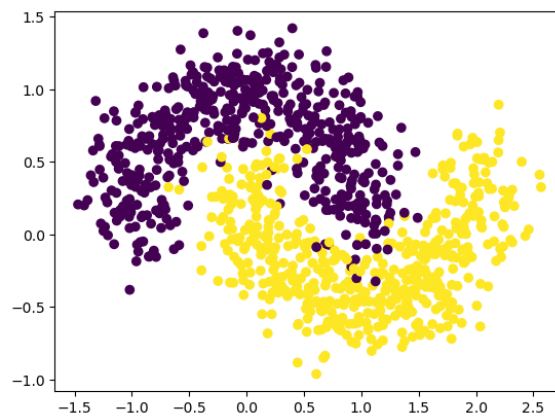


FIGURE 7. A Non-Linearly seperable dataset. (Author's Own)

We will train a linear SVC to attempt to classify this data.

```
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1)
svm1 = svm.SVC(kernel='linear').fit(X1_train,y1_train)

xx, yy = np.meshgrid(np.arange(X1_train[:,0].min()-1, X1_train[:,0].max()+1, 0.1),
np.arange(X1_train[:,1].min()-1, X1_train[:,1].max()+1, 0.1))

Z = svm1.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.scatter(X1_train[:,0], X1_train[:,1], c = y1_train, marker='o')
plt.contourf(xx,yy,Z, cmap=plt.cm.coolwarm, alpha=0.4)
plt.show()
```

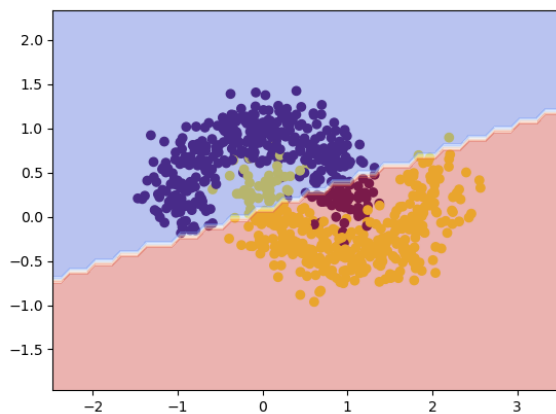


FIGURE 8. A Linear SVC trained on the above dataset. (Author's Own)

As is to be expected, the Linear SVC fares very poorly. It is not capturing the nature of the data at all.

4. THE KERNEL TRICK

How do we resolve the above problem? One way to do so is by adding features. Consider the following diagram below.

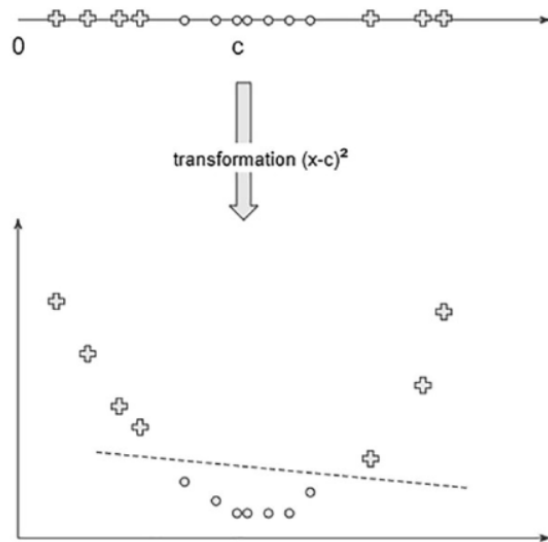


FIGURE 9. Transforming a dataset (Adding features). [4]

The 1-dimensional dataset, before the transformation, is not linearly separable. The 2-dimensional dataset, however, is linearly separable, and a separating hyperplane can be seen on the diagram. This is what we are doing when we change the kernel

of the SVM, we are adding features. An example of a kernel, that we will use below, is the *radial basis function*, given by $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$. We illustrate various kernels with the following python code.

```
rbf_svc = svm.SVC(kernel='rbf', gamma=5, C=1).fit(X1_train, y1_train)
poly_svc = svm.SVC(kernel='poly', degree=3, C=5).fit(X1_train, y1_train)
lin_svc = svm.LinearSVC(C=1).fit(X1_train, y1_train)

titles1 = [ 'SVC with RBF kernel', 'SVC with polynomial (degree 3) kernel.' ]
kernels = ['RBF', 'Polynomial']
xx, yy = np.meshgrid(np.arange(X1_train[:,0].min()-1, X1_train[:,0].max()+1, 0.1),
                     np.arange(X1_train[:,1].min()-1, X1_train[:,1].max()+1, 0.1))

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(30, 15), sharey=True)
for i, clf in enumerate((rbf_svc, poly_svc)):
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    axes[i].scatter(X1_train[:,0], X1_train[:,1], c = y1_train, marker='o')
    axes[i].scatter(clf.support_vectors_[0], clf.support_vectors_[1],
                    s=75, edgecolor='k',
                    facecolor='k', label='{} support vectors'.format(kernels[i]))
    axes[i].contourf(xx,yy,Z, cmap=plt.cm.coolwarm, alpha=0.4)
    axes[i].legend()
    axes[i].set_title(titles1[i])

plt.show()
```

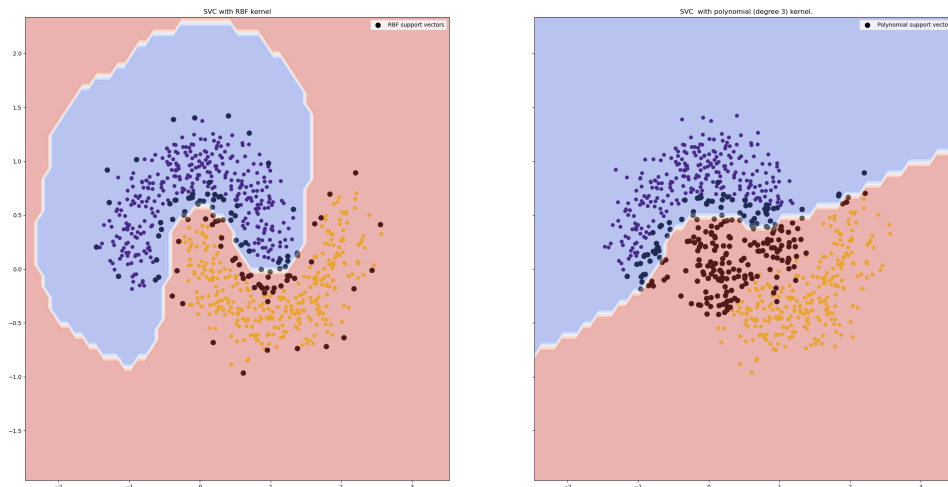


FIGURE 10. SVM's trained with various Kernels. The left is the RBF, the right is a polynomial kernel of degree three. (Author's Own)

```
from sklearn.calibration import cross_val_predict
```

```

y_train_pred = cross_val_predict(rbf_svc, X_train, y_train, cv=3)
cm1 = metrics.confusion_matrix(y_train, y_train_pred)
sns.heatmap(cm1, annot=True, fmt='.0f')

```

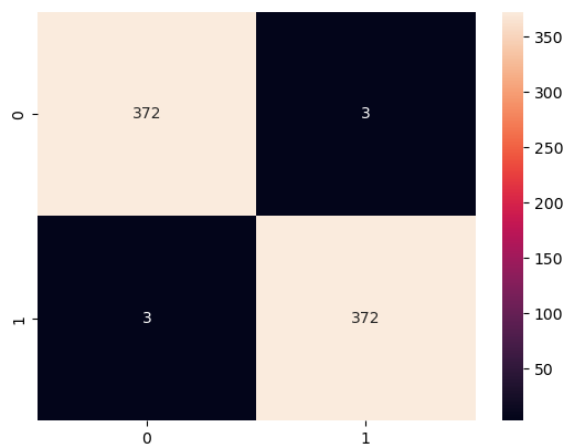


FIGURE 11. The confusion matrix for the above SVM's (Author's Own).

5. MULTILABEL CLASSIFICATION

A benefit of SVM's is that they natively support multilabel classification. We can see this with the iris dataset.

```

from sklearn.datasets import load_iris
import numpy as np

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
df['target'] = iris.target
X2 = df.iloc[:, :2].to_numpy()
y2 = df.iloc[:, 5].to_numpy()

plt.scatter(X2[:, 0], X2[:, 1], c=y2)
plt.xlabel('sepal width (cm)')
plt.ylabel('sepal length (cm)')
plt.legend()
plt.show()

```

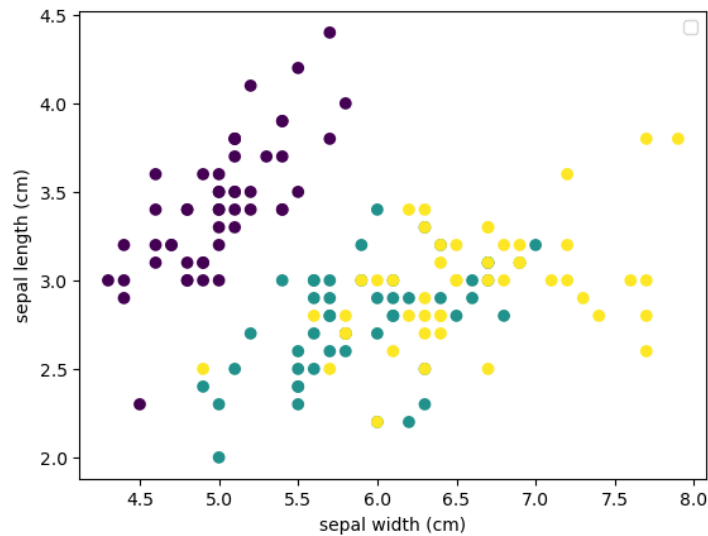



FIGURE 12. The Iris Dataset

```

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2)
lin_svc = svm.SVC(kernel='linear', gamma=1, C=1).fit(X2_train, y2_train)

xx, yy = np.meshgrid(np.arange(X2_train[:,0].min()-1, X2_train[:,0].max()+1, 0.1),
                     np.arange(X2_train[:,1].min()-1, X2_train[:,1].max()+1, 0.1))

Z = lin_svc.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.scatter(X2_train[:,0], X2_train[:,1], c = y2_train, marker='o')
plt.scatter(lin_svc.support_vectors_[0],
            lin_svc.support_vectors_[1],
            s=75, facecolor = 'none',
            edgecolor = 'r', label = 'Support Vectors')

plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.4)
plt.title('Linear SVC on iris dataset.')
plt.legend()
plt.xlabel('sepal width (cm)')
plt.ylabel('sepal length (cm)')
plt.show()

```

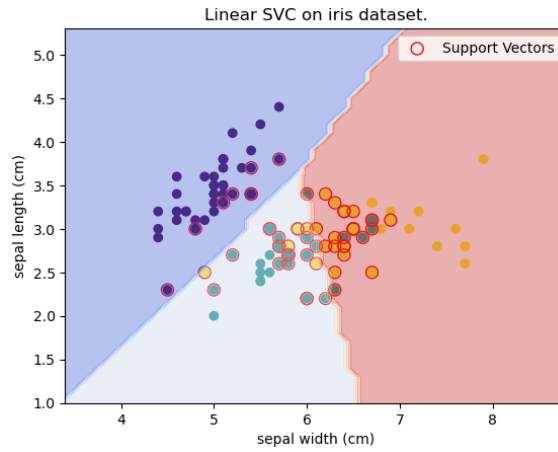


FIGURE 13. Training an SVM for the Iris Dataset. (Author's Own)

6. SUPPORT VECTOR REGRESSION

The way to use SVMs for regression, linear or nonlinear, is to reverse the objective. Instead of trying to fit the maximum hyperplane between two classes while limiting margin violations, SVM regression tries to fit as many instances as possible on the street while limiting margin violations - the width of the street is controlled by a hyperparameter, ε .

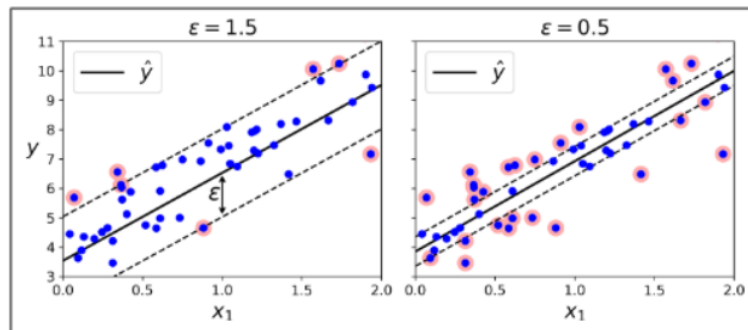


FIGURE 14. The Idea of Support Vector Regression [3].

We illustrate this in python. First, we generate data.

```
X3 = np.sort(5* np.random.rand(100, 1), axis=0)
y3 = np.cos(X3).ravel()
plt.scatter(X3, y3, c='b')
```

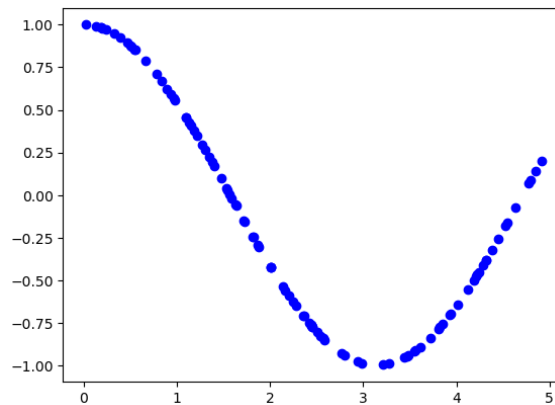


FIGURE 15. Generating data for Support Vector Regression.

Next, we define various support vector machines for doing regression on the above dataset.

```
svr_rbf = svm.SVR(kernel='rbf',C=100,gamma=0.1,epsilon=0.1).fit(X3,y3)
svr_lin= svm.SVR(kernel='linear', C=100, gamma='auto').fit(X3,y3)
svr_poly = svm.SVR(kernel='poly', C=100, gamma='auto',
    degree=3,
    epsilon=0.1,
    coef0=1).fit(X3,y3)
```

```
svrs = [svr_rbf, svr_lin, svr_poly]
```

Finally, we plot what the various SVR's tell us.

```
kernel_label = ["RBF", "Linear", "Polynomial"]
model_color = ["firebrick", "darkcyan", "violet"]
fig, axes = plt.subplots(nrows=1,ncols=3, figsize=(15,10), sharey=True)
for i, svr in enumerate(svrs):
    axes[i].plot(
        X3, svr.predict(X3),
        lw = 2,
        label = "{} model".format(kernel_label[i]),
        c = model_color[i]
    )
    axes[i].scatter(
        X3, y3, facecolor="none", edgecolor = model_color[i], s=75,
        label = "training data"
    )
    axes[i].scatter(
        X3[svr.support_], y3[svr.support_], s=75, facecolor = "none", edgecolor = 'k',
        label = "{} model support vectors".format(kernel_label[i]),
    )
    axes[i].legend()
```

```

    loc = 'upper center',
    fancybox= True,
    shadow = True
)

```

```
fig.suptitle("Support Vector Regression Models", fontsize=14)
```

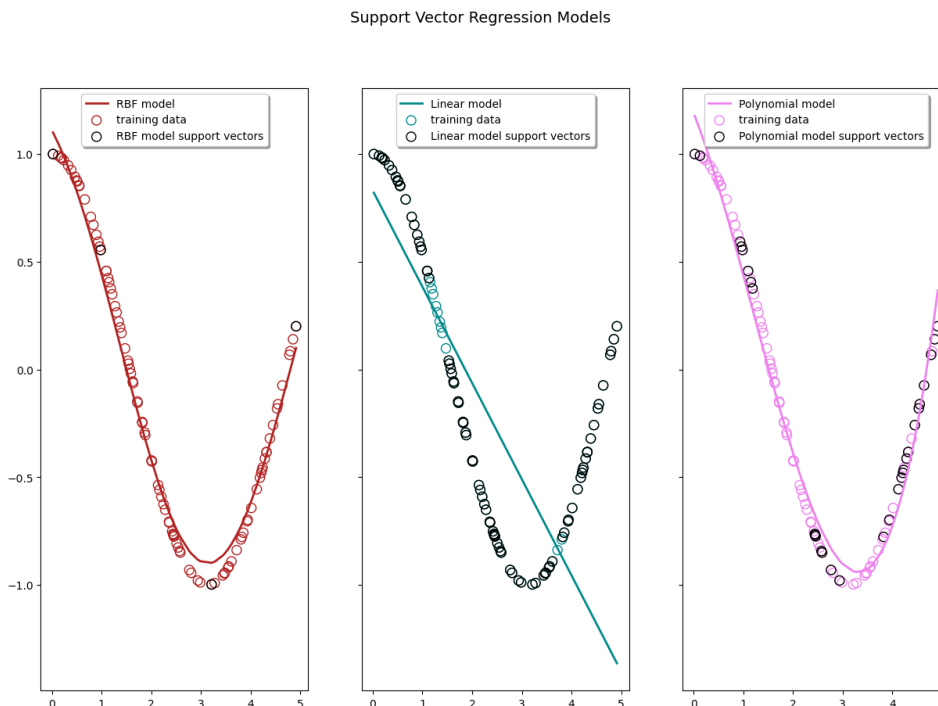


FIGURE 16. The various Support Vector Machines. (Author's Own)

7. CLOSING REMARKS

One way to improve SVM performance is hyperparameter tuning. Each SVM comes with certain hyperparameters that affect the way they perform. One that we've met already is the kernel. Another that is applicable to all SVM's is the value of c - the amount of margin violations we allow. For support vector regression, we consider how wide we want our "street" to be - this epsilon is another hyperparameter. Sci-kit learn comes with built in functions that allow for optimisation of hyperparameters, such as `GridSearchCV`, which was seen before - it performs an exhaustive search through a set of hyperparameters that you decide and returns the optimal model. Moreover, we can use any function we like as a kernel, as long as it can be expressed as an inner product in another space.

The main issue with SVM's is that they simply take a long time to train. To illustrate, suppose an oracle reveals to you which points are the optimal support vectors and which aren't. Suppose we have R support vectors, finding the coefficients

of the hyperplane then amounts to solving a linear system of R equations, and this is well known to require a number of operations proportional to R^3 . The number of support vectors grows linearly with the number of examples, so when the number of datapoints is large, the minimisation problem has computational complexity at least $O(n^3)$ [2].

SVM's are hugely applicable, we have seen here that they can be used for both classification and regression, but there is also a reformulation that is used for outlier detection, for example. Moreover, they can be tuned very finely, as you can choose your own kernel, for example, and each kernel has a large amount of hyperparameters. Moreover, they perform very well on small datasets, however they take a long time to train on large datasets. (Training an SVM on the aforementioned MNIST dataset took well over 30minutes). So, to conclude, SVM's are a very powerful, very precise tool, useful mainly in situations where you have a small to medium-sized dataset.

REFERENCES

- [1] URL: https://commons.wikimedia.org/wiki/File:Svm_max_sep_hyperplane_with_margin.svg.
- [2] Léon Bottou, Chih-Jen Lin et al. 'Support vector machine solvers'. In: *Large scale kernel machines* 3.1 (2007), pp. 301–320.
- [3] Aurélien Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. " O'Reilly Media, Inc.", 2022.
- [4] Ashwin Pajankar and Aditya Joshi. *Hands-on Machine Learning with Python: Implement Neural Network Solutions with Scikit-learn and PyTorch*. apress, 2022.